

Attendee Handout — Submitting AI Jobs to the Anvil Supercomputer

AI for All Workshop - Beginner Track 1 - September 15, 2026 - 1:00-2:30 PM CT

Keep this handout — every command you need today is here, plus what to try next.

What you're working with

Anvil (Purdue University, NSF-funded, ACCESS/NAIRR resource): 1,000+ compute nodes, 128 cores/node (AMD EPYC “Milan”), 16 nodes x 4x NVIDIA A100, 21 nodes x 4x NVIDIA H100, Slurm scheduler. You never run work on the login node — you submit **jobs** to **queues** (Slurm calls them *partitions*).

Partition	What it's for	Limit
debug	quick tests (we use this)	2 nodes, 2 hrs
gpu-debug	quick GPU tests (we use this)	1 node, 2 GPUs, 30 min
shared	default CPU jobs	128 cores, 96 hrs
wholenode / wide	exclusive multi-node CPU	96–128 hrs / 12 hrs
highmem	1 TB RAM nodes	48 hrs (charges 4x)
gpu	A100 production jobs	48 hrs
ai	H100 production jobs	48 hrs

Two lines in every job script are mandatory on Anvil:

```
#SBATCH -A <your-account>      # from: mybalance
#SBATCH -p <partition>         # from: showpartitions
```

Connect (Lab 1)

```
ssh <your-anvil-username>@anvil.rcac.purdue.edu
```

- Your Anvil username starts with x- (derived from your ACCESS ID).
- Passwords don't work — SSH keys only. No key yet? Log in through the browser instead: <https://ondemand.anvil.rcac.purdue.edu> -> *Clusters* -> *Shell access* (everything today works there too).
- If your SSH says failed to map user <you>@access-cli.org: your allocation isn't set up yet — see “Getting your own allocation” below, and pair up with a neighbor for now.

Once you're in:

```
mybalance      # your allocation account string -> this is your -A value
showpartitions # all queues, cores free/total, time limits
sfeatures      # node types: a* = CPU, g* = A100 (gpu:4), h* = H100 (gpu:4)
```

First job (Lab 2)

Files: hello.py + hello.sbatch. Open hello.sbatch, replace <ACCOUNT> with your mybalance account string, then:

```
sbatch hello.sbatch      # -> "Submitted batch job 123456"
squeue -u $USER           # watch: PENDING -> RUNNING -> (gone)
ls slurm-*.out            # your output file
cat slurm-<jobid>.out     # "Hello from Anvil!" + which node ran it
```

The output file shows SLURM_JOB_NODELIST — proof the job ran on a **different machine** than the one you typed on. That's the whole model in one job.

A real AI job on CPU (Lab 3)

Files: `train_digits.py` + `train-digits-cpu.sbatch`. Trains a digits classifier (scikit-learn) on 1,797 handwritten-digit images — no download, dataset ships with the library, runs in about a minute.

```
sbatch train-digits-cpu.sbatch
squeue -u $USER ; ls
cat digits-cpu_<jobid>.out      # test accuracy ~ 0.98
cat digits_metrics.json         # machine-readable results
```

Try after: change `--cpus-per-task=16` to 8, resubmit, compare elapsed time in the output.

Your first GPU job (Lab 4)

Files: `train_gpu.py` + `train-gpu.sbatch`. Trains a small neural network (PyTorch) and prints `nvidia-smi` so you can see the **A100 your job landed on**. Partition `gpu-debug` (30-min limit — made for testing).

```
sbatch train-gpu.sbatch
squeue -u $USER                # node name starts with "g" = A100 node
cat slurm-<jobid>.out           # nvidia-smi + per-epoch training times
```

The two new lines vs. Lab 3: `--gres=gpu:1` (asking for one GPU) and `-p gpu-debug`.

Stretch: one script, many experiments

```
sbatch --array=0-4 array.sbatch  # 5 jobs in one command: a C-value sweep
ls slurm-array_*.out             # one output per array task
```

Each task reads `SLURM_ARRAY_TASK_ID` and trains with a different `C` — the hyperparameter-sweep pattern real research jobs use every day.

Job management cheat sheet

Command	What it does
<code>sbatch <script></code>	Submit a batch job
<code>squeue -u \$USER</code>	Your jobs in the queue (ST column: PD = waiting, R = running)
<code>squeue -j <jobid></code>	One job's status
<code>scancel <jobid></code>	Cancel a job (your own only)
<code>sacct -j <jobid></code>	A finished job's accounting: state, elapsed, max memory
<code>sinfo -p debug</code>	Partition status
<code>srun --pty -A <acct> -p debug -t 00:30:00 --nodes=1 --ntasks=1 /bin/bash</code>	Interactive shell on a compute node
<code>mybalance</code>	Your remaining allocation (SUs)
<code>ssh <user>@anvil.rcac.purdue.edu</code>	Log in
<code>module load conda</code>	Load Anaconda Python
<code>module spider <name></code>	Search all available software
<code>module list / module purge</code>	What's loaded / unload everything

Troubleshooting

Symptom	Cause -> fix
error: invalid account specified / account errors	Missing or wrong -A. Run <code>mybalance</code> , copy the string exactly.
error: Invalid partition name	Typo in -p. Run <code>showpartitions</code> .
Job stuck in PENDING (PD)	Queue busy (Resources) or time limit exceeds partition max (<code>squeue -j <id> -o "%Q"</code> shows the reason).
slurm-*.out empty	Job still queued — check <code>squeue</code> . Output flushes when the job runs.
ModuleNotFoundError: No module named 'sklearn'/torch	Forgot <code>module load conda</code> (or the PyTorch module) in the script.
CUDA error / no GPU in output	Script didn't request <code>--gres=gpu:1</code> or wrong partition (CPU-only).
Can't SSH	SSH key not set up -> use OnDemand in the browser (link above).

Getting your own allocation (after today)

- **ACCESS credits** -> transfer to Anvil: <https://allocations.access-ci.org>
- **NAIRR Pilot** -> apply through <https://www.nairrpilot.org> — when your proposal is accepted for Anvil, the allocation is created automatically (no credit transfer)
- Docs: <https://docs.rcac.purdue.edu/userguides/anvil/> (Job Submission page is the one we used today)
- Help: rcac-help@purdue.edu — mention “AI for All workshop” for context.

Rule of the road: nothing heavier than `python hello.py` on the login node. Everything else goes through `sbatch` (or `srun -pty` for interactive work).